Software Requirements Specification

CodeQuest: RPG Coding Adventure

Frontend-only RPG-style DSA/Coding Practice Web Application | Version 1.0 | October 2026 | Structure based on IEEE-style SRS practice (ISO/IEC/IEEE 29148) | Prepared by: [Team / Student Names]

1. INTRODUCTION

1.1 Purpose

This SRS defines the functional, non-functional, interface, data, testing and scope requirements of **CodeQuest: RPG Coding Adventure**. It is the common reference for students, developers, designers, testers and faculty evaluators during design, development, testing and evaluation. It specifies only what the current version will do; ideas for later versions are listed separately as future enhancements.

1.2 Project Scope

CodeQuest turns DSA and coding practice into an RPG-style adventure. The learner becomes a hero who enters quest realms, solves predefined coding/DSA challenges, earns XP, levels up, unlocks new quests, collects badges, maintains a daily streak and tracks progress. The software runs entirely in the browser as a frontend-only application. Its goal is to combine **learning + gaming** so that practice becomes engaging and less monotonous.

1.3 Product Overview

CodeQuest is a set of HTML5 pages styled with CSS3 and driven by Vanilla JavaScript. Challenges are stored inside JavaScript data structures. The player flows from a Landing Page to a Dashboard, then to the Quest Map, a Challenge Screen, a Result Screen and back to updated Progress. Progress may be saved in browser localStorage. The UI is designed in Figma and the source code is maintained and hosted on GitHub.

1.4 Intended Audience

Audience	Use of this document
Students / Coding and DSA learners	Understand what the platform offers and how to use it

Audience	Use of this document
Faculty / Evaluators	Evaluate completeness, quality and scope for submission and viva
Project developers	Implement the HTML, CSS and JavaScript according to requirements
UI/UX designers	Create wireframes, design system and prototype in Figma
Testers	Derive test cases and verify requirements

1.5 Definitions, Acronyms and Abbreviations

Term	Meaning
DSA	Data Structures and Algorithms
RPG	Role-Playing Game: a game where a character grows through tasks and levels
UI / UX	User Interface / User Experience
XP	Experience Points earned by completing challenges
SRS	Software Requirements Specification
HTML / CSS	HyperText Markup Language / Cascading Style Sheets
JavaScript	Scripting language used for all logic (Vanilla JS means no library or framework)
Quest	A themed group of challenges, one per DSA category
Challenge	A single question/problem the player solves
Streak	Number of consecutive days with at least one completed challenge
Badge	Reward shown when an achievement condition is met
localStorage	Browser storage that keeps small key-value data on the user's device

Term	Meaning
FR / NFR / UC / TC	Functional Requirement / Non-Functional Requirement / Use Case / Test Case

1.6 References

- IEEE Std 830-1998, *Recommended Practice for Software Requirements Specifications*, and its successor ISO/IEC/IEEE 29148:2018.
- W3C HTML5 and CSS specifications; ECMAScript Language Specification.
- MDN Web Docs (HTML, CSS, JavaScript, Web Storage).
- W3C Web Content Accessibility Guidelines (WCAG) 2.1.
- Figma Help Center and GitHub Docs (including GitHub Pages).

2. OVERALL DESCRIPTION

2.1 Product Perspective

CodeQuest is a self-contained, **frontend-only web application**. All pages, challenge data, validation, XP/level calculation and progress logic run in the user's browser. **There is no backend, no database and no API in the current version.** Optional progress persistence uses browser localStorage only. The hosted copy is served as static files through GitHub Pages.

2.2 Product Functions

Landing page; player introduction/profile; RPG dashboard; quest selection; DSA category selection; difficulty selection; coding challenge interface; answer validation; XP calculation; level progression; progress tracking; achievements/badges; streak indicators; locked/unlocked quests; result/feedback screen; navigation; responsive interface; local progress storage; reset progress. A leaderboard-style panel using fixed mock data is optional and informational only.

2.3 User Classes

User class	Description and key activities
Learner / User (primary)	Starts the adventure, explores quests, attempts challenges, earns XP, unlocks levels and quests, views progress and badges

User class	Description and key activities
Project Evaluator / Faculty (secondary)	Opens the application to review functionality, UI and workflow; uses the same interface as a learner (no separate role)

No admin or backend role exists in the current scope.

2.4 Operating Environment

- Devices: desktop, laptop, tablet and mobile phone.
- Browsers: current versions of Google Chrome, Microsoft Edge, Mozilla Firefox (Safari and other modern browsers as available).
- Operating systems: Windows, macOS, Linux, Android, iOS (any OS with a modern browser).

2.5 Design and Implementation Constraints

- Frontend-only architecture; **HTML5** for structure, **CSS3** for styling/animations/responsiveness, **Vanilla JavaScript** for all logic.
- **Figma** for UI/UX design and prototype; **GitHub** for version control and hosting.
- No backend, no database, no external framework or external API.
- Challenge content must be predefined inside the project files.

2.6 Assumptions and Dependencies

- The user has a modern browser with JavaScript enabled.
- Challenges are predefined in JavaScript arrays/objects.
- Progress may be stored locally in the browser; clearing browser data removes it.
- Internet is needed only to open the hosted GitHub Pages version; a local copy works offline.
- The device date is used for streak calculation.

3. SYSTEM FEATURES / FUNCTIONAL REQUIREMENTS

All requirements below are **current scope** unless explicitly marked otherwise.

FR-01 Landing Page. *Description:* Entry page introducing CodeQuest with a "Start Adventure" call to action. *Input:* Click on Start Adventure / navigation link. *Processing:* Check whether a saved player exists; route to profile creation or dashboard. *Output:*

Landing content, then redirect to the correct screen. *Acceptance:* Page loads with title, tagline, feature highlights and a working Start button; returning player goes straight to dashboard.

FR-02 Player Introduction/Profile. *Description:* New player enters a hero name and picks an avatar/class icon from predefined options. *Input:* Name (2-20 characters), avatar choice. *Processing:* Validate and sanitize name; create player object (Level 1, 0 XP, streak 0). *Output:* Profile card with name, avatar, level, XP. *Acceptance:* Empty/invalid names show an error; valid input creates the profile and opens the dashboard.

FR-03 Dashboard. *Description:* Central RPG hub. *Input:* Navigation to dashboard. *Processing:* Read player and progress data; compute summary values. *Output:* Profile card, XP/level bar, streak, completed count, quest shortcuts, recent badges. *Acceptance:* All values match stored data and update after each challenge.

FR-04 Quest Selection. *Description:* Quest map listing 8 quests (one per DSA category) in order. *Input:* Click on a quest card. *Processing:* Check unlock status of the quest. *Output:* Open quest if unlocked; "Locked" message with unlock condition otherwise. *Acceptance:* Locked quests cannot be opened; unlocked quests open correctly.

FR-05 DSA Category Selection. *Description:* Player views categories (Arrays, Strings, Searching, Sorting, Linked List, Stack, Queue, Basic Algorithms) with completion counts. *Input:* Category/quest click. *Processing:* Filter challenges by category. *Output:* Category screen with its challenges and progress. *Acceptance:* Only challenges of the selected category are shown.

FR-06 Difficulty Selection. *Description:* Player chooses Easy, Medium or Hard within a quest. *Input:* Difficulty button. *Processing:* Check tier unlock rules (Section 15); filter challenges. *Output:* List of challenges of that difficulty with XP value; locked tiers shown disabled. *Acceptance:* Each tier shows its XP reward and correct lock state.

FR-07 Coding Challenge. *Description:* Screen showing title, description, difficulty, XP reward, question and an input area (options, text input or code/answer textarea). *Input:* Player's answer; Hint/Back actions. *Processing:* Load challenge by ID from challenge data. *Output:* Complete challenge screen. *Acceptance:* All fields display correctly; missing challenge shows an error message.

FR-08 Challenge Validation. *Description:* Checks the submitted answer against the predefined solution. *Input:* Submitted answer. *Processing:* Trim, normalize case/whitespace, compare with accepted answers or required keywords; reject empty input. *Output:* Correct/Incorrect result with message. *Acceptance:* Correct answers pass, wrong ones fail, empty input is rejected without penalty. No real code compilation is performed.

FR-09 XP Reward System. *Description:* Awards XP on first successful completion. *Input:* Validation success. *Processing:* Add difficulty-based XP (Easy 10, Medium 25, Hard 50) only if the challenge was not completed earlier. *Output:* Updated XP total and "+XP" indicator. *Acceptance:* XP is added once per challenge; repeat solves give 0 XP.

FR-10 Level Progression. *Description:* Level derived from total XP. *Input:* Updated XP. *Processing:* Level = floor(XP / 100) + 1, capped at Level 10. *Output:* New level, progress bar, level-up animation/message. *Acceptance:* Level changes exactly at thresholds; bar shows percentage to next level.

FR-11 Progress Tracking. *Description:* Shows overall and per-quest progress. *Input:* Open progress section. *Processing:* Calculate completed/total challenges and percentages. *Output:* Progress bars, counts, XP earned, streak. *Acceptance:* Percentages are accurate after every completion.

FR-12 Achievement/Badge System. *Description:* Awards badges when conditions in Section 16 are met. *Input:* Progress change events. *Processing:* Evaluate achievement rules; mark newly unlocked. *Output:* Badge unlock notification and Achievements page (earned vs locked). *Acceptance:* Each badge unlocks once and only when its condition is satisfied.

FR-13 Streak System. *Description:* Tracks consecutive days of activity. *Input:* Challenge completion date. *Processing:* Same day = no change; next day = +1; gap of more than one day = reset to 1. *Output:* Streak counter on dashboard. *Acceptance:* Streak updates correctly for same-day, consecutive and missed-day cases.

FR-14 Locked/Unlocked Quest System. *Description:* Controls access to quests and difficulty tiers. *Input:* Progress data. *Processing:* Apply unlock rules (Section 15). *Output:* Locked cards with padlock icon and condition text; unlocked cards enabled. *Acceptance:* Next quest unlocks automatically when its condition is met.

FR-15 Challenge Completion. *Description:* Finalizes a solved challenge. *Input:* Successful validation. *Processing:* Record challenge ID as completed, update XP, level, streak, unlocks, badges, and save. *Output:* Transition to result screen. *Acceptance:* All derived values update in one consistent step.

FR-16 Result/Feedback Screen. *Description:* Summarizes the attempt. *Input:* Completion/failure outcome. *Processing:* Prepare message, XP gained, level-up and unlock/badge notices. *Output:* Success or retry screen with buttons: Next Challenge, Retry, Back to Quest, Dashboard. *Acceptance:* Feedback is shown after every submission; buttons navigate correctly.

FR-17 Navigation. *Description:* Persistent navigation bar (Home, Dashboard, Quests, Progress, Achievements) and Back buttons. *Input:* Link/button click. *Processing:* Show requested section; block invalid states. *Output:* Correct screen with active-link highlight.

Acceptance: All links work; locked or invalid destinations redirect safely.

FR-18 Responsive Interface. *Description:* Layout adapts to desktop, tablet and mobile. *Input:* Screen width/orientation change. *Processing:* CSS media queries, flexible grid/flexbox. *Output:* Readable, usable layout without horizontal scrolling. *Acceptance:* Verified at about 1366 px, 768 px and 375 px widths.

FR-19 Local Progress Storage. *Description:* Saves and restores player data in localStorage (if available). *Input:* Any progress change; application start. *Processing:* Serialize to JSON; load and validate on start. *Output:* Progress persists across sessions. *Acceptance:* Reload restores state; if storage is unavailable or data is corrupt, the app runs in session-only mode with a notice.

FR-20 Reset/Restart Progress. *Description:* Lets the player erase progress. *Input:* Reset button and confirmation. *Processing:* Confirm, delete stored data, restore defaults. *Output:* Fresh state at Level 1 and profile screen. *Acceptance:* Nothing is reset without confirmation; all values return to defaults.

Future (not current scope): server-side saving, login, global leaderboard, real code execution.

4. NON-FUNCTIONAL REQUIREMENTS

ID	Category	Requirement
NFR-01	4.1 Performance	Pages load within 3 seconds on a normal connection; screen changes, validation and XP updates respond within 1 second; total project size stays small (no large media files); animations run smoothly without freezing the browser.
NFR-02	4.2 Usability	A new learner can start and complete a first challenge without instructions; consistent layout, labels and buttons; clear feedback messages; maximum three clicks from dashboard to a challenge.
NFR-03	4.3 Accessibility	Semantic HTML5 elements; keyboard-operable controls with visible focus; text contrast aligned with WCAG 2.1 AA; alt text for images; labels for inputs; status is never conveyed by colour alone (icons/text accompany locked/completed states).

ID	Category	Requirement
NFR-04	4.4 Responsiveness	Fluid layouts using flexbox/grid and media queries for mobile (below 600 px), tablet (600-1024 px) and desktop (above 1024 px); touch-friendly buttons (about 44 px); no horizontal scrolling.
NFR-05	4.5 Reliability	Application must not crash on invalid input; XP is never awarded twice for one challenge; corrupt or missing stored data falls back to defaults; calculations must be deterministic.
NFR-06	4.6 Maintainability	Separate HTML, CSS and JS files; modular JavaScript functions with meaningful names; challenge data kept apart from logic; commented code; consistent naming and folder structure.
NFR-07	4.7 Compatibility	Works on current versions of Chrome, Edge and Firefox on Windows, macOS and Linux, and on mobile browsers; only widely supported web features are used.
NFR-08	4.8 Security	No passwords or sensitive data; input validation and escaping; no unsafe dynamic HTML insertion (see Section 18).
NFR-09	4.9 Scalability	New challenges, categories and badges can be added by editing data arrays without changing core logic; the structure allows later addition of a backend (future scope only).
NFR-10	4.10 Portability	Runs from any static host or by opening the files locally; no installation or build step; no server configuration.

5. USER INTERFACE REQUIREMENTS

Visual direction. The interface should be game-like, modern, engaging, clear and student-friendly: a dark fantasy-tech theme with bright accent colours, rounded cards, progress bars, XP and level indicators, badges, padlock icons for locked content, consistent typography, clear buttons and purposeful animations (XP fill, level-up pop, badge glow) that can be reduced for users who prefer less motion. No external UI library is used.

No.	Screen/Component	Requirements
1	Landing Page	Hero section with title, tagline and prominent "Start Adventure" button; short feature highlights (Quests, XP, Badges, Streaks); footer.
2	Navigation Bar	Fixed top bar with logo and links (Home, Dashboard, Quests, Progress, Achievements); active-page highlight; collapses to a menu button on mobile.
3	Dashboard	Grid of cards: profile card, XP/level bar, streak card, completed challenges, current quest, recent badges, "Continue Adventure" button.
4	Player Profile	Avatar, hero name, level, total XP, rank title (for example Novice Coder), edit-name option, reset-progress option.
5	Quest Map	Eight quest cards in order, each showing icon, name, completion count/bar and Locked/Unlocked state with unlock hint.
6	Challenge Screen	Title, category and difficulty tag, XP reward, description, question, hint button, answer area, Submit and Back buttons.
7	Code/Input Area	Monospace textarea or text input (or option buttons for MCQ); character limit; clear/reset button; visible validation message; no code execution.
8	XP/Level Display	Level number, XP count (for example 240 / 300), animated progress bar and "+XP" popup.
9	Progress Section	Overall percentage, per-quest progress bars, counts by difficulty, streak value.
10	Achievement Section	Grid of badge cards; earned badges coloured, locked ones greyed with condition text.
11	Result Screen	Success/failure banner, explanation, XP gained, level-up/unlock/badge notices, action buttons.
12	Mobile Responsive Layout	Single-column stacking, hamburger navigation, full-width buttons, readable font sizes (minimum 16 px body text).

6. UI/UX REQUIREMENTS

Use of Figma. Figma is used before coding to design and review the interface: wireframes for all screens; a user-flow diagram (Landing to Dashboard to Quest to Challenge to Result); page layouts for desktop, tablet and mobile; reusable components (buttons, cards, progress bars, badges, nav bar); colour palette and typography styles; a clickable prototype for walkthrough and feedback; and responsive frames for each breakpoint.

CodeQuest Design System (simple).

Element	Specification
Primary colour	Deep navy / dark purple background (for example #12122A)
Accent colours	Gold (XP, #F5B700), Teal (success, #19C3A6), Red (error, #E5484D), Violet (buttons, #7C5CFF)
Typography	One heading font and one readable body/monospace font from system or locally included fonts; heading 28-40 px, body 16 px, code 14-16 px
Spacing and shape	8 px spacing grid; 12 px rounded corners; soft shadows
Components	Primary/secondary/disabled buttons, quest card (locked/unlocked), progress bar, badge, tag (difficulty), alert message
Motion	200-400 ms transitions; XP bar fill; level-up pop-up
UX principles	Consistency, immediate feedback, visible progress, minimal clutter, clear locked/unlocked cues

7. SYSTEM WORKFLOW

Landing Page → Start Adventure → Dashboard → Select Quest → Select DSA Topic → Select Difficulty → Read Challenge → Submit/Check Answer → Validation → Earn XP → Update Level/Progress → Unlock Next Quest → Continue Adventure

Stage	Explanation
1. Landing Page	Visitor sees the project introduction and the Start button.

Stage	Explanation
2. Start Adventure	New player creates a hero profile; returning player skips ahead.
3. Dashboard	Player sees level, XP, streak, progress and quest shortcuts.
4. Select Quest	Player chooses an unlocked quest from the quest map.
5. Select DSA Topic	Quest category (for example Arrays) is confirmed and its challenges are listed.
6. Select Difficulty	Player picks Easy, Medium or Hard (subject to unlock rules).
7. Read Challenge	Challenge text, XP reward and optional hint are displayed.
8. Submit/Check Answer	Player enters an answer and submits.
9. Validation	Empty input rejected; answer compared with accepted solution.
10. Earn XP	If correct and new, difficulty-based XP is added.
11. Update Level/Progress	Level, progress bars, streak and badges are recalculated and saved.
12. Unlock Next Quest	Unlock rules are re-evaluated; new content is opened.
13. Continue Adventure	Player moves to next challenge, quest or dashboard.

8. USE CASES

ID	Name	Actor	Goal	Preconditions	Main Flow	Alternative Flow	Postconditions
UC-01	Start Adventure	Learner	Begin the game	App opened	1. Open landing page. 2. Click Start. 3. Enter name, choose avatar. 4. Confirm.	Invalid name: error and retry. Saved player: go straight to dashboard.	Profile created; dashboard open

ID	Name	Actor	Goal	Preconditions	Main Flow	Alternative Flow	Postconditions
UC-02	View Dashboard	Learner	See status overview	Profile exists	1. Open Dashboard. 2. System loads data. 3. Cards display values.	Data missing: defaults shown.	Dashboard displayed
UC-03	Select Quest	Learner	Choose a quest	Dashboard/Quest Map open	1. Open quest map. 2. Click unlocked quest. 3. View category and difficulties.	Quest locked: lock message shown.	Quest screen open
UC-04	Attempt Challenge	Learner	Read and work on a challenge	Quest and difficulty selected	1. Pick challenge. 2. Read question. 3. Enter answer.	Challenge missing: error and return. Hint requested: hint shown.	Challenge ready for submission
UC-05	Submit Answer	Learner	Get answer checked	Answer entered	1. Click Submit. 2. System validates. 3. Result shown.	Empty answer: prompt. Incorrect: feedback, retry allowed.	Attempt evaluated
UC-06	Earn XP	Learner	Receive reward	Answer correct	1. System checks first completion. 2. Adds XP. 3. Updates bar.	Already completed: 0 XP.	XP total updated and saved
UC-07	Unlock Level	Learner	Advance in game	XP updated	1. System recalculates level. 2. Re-evaluates quest/tier unlocks. 3. Shows notice.	No threshold reached: no change.	New level/quest available
UC-08	View Progress	Learner	Review learning progress	Profile exists	1. Open Progress. 2. View bars, counts, streak.	No activity: zero values shown.	Progress displayed
UC-09	View Achievements	Learner	See badges	Profile exists	1. Open Achievements. 2. View earned/locked badges and conditions.	None earned: all locked.	Badge status displayed
UC-10	Continue Quest	Learner	Keep playing	Challenge completed	1. Click Next Challenge or Back to Quest. 2. Next challenge loads.	All done: completion message and dashboard link.	Player in next challenge/quest

9. USE CASE DIAGRAM

System boundary: CodeQuest (frontend-only web application). **Primary actor:** Learner/User (placed outside the boundary on the left). No other actors are required.

Use cases inside the boundary: Start Adventure, View Dashboard, Select Quest, Select Challenge, Attempt Challenge, Submit Answer, Earn XP, Track Progress, View Achievements, Unlock Level.

Associations and relationships:

- Learner is associated with Start Adventure, View Dashboard, Select Quest, Select Challenge, Attempt Challenge, Submit Answer, Track Progress and View Achievements.
- Select Challenge **<<include>>** Select Quest (a challenge is chosen inside a quest).
- Submit Answer **<<include>>** Attempt Challenge.
- Earn XP **<<extend>>** Submit Answer (only when the answer is correct).
- Unlock Level **<<extend>>** Earn XP (only when an XP threshold is reached).
- Track Progress and View Achievements read the data updated by Earn XP and Unlock Level.

The diagram can be drawn with any UML drawing tool or in Figma using ovals for use cases, a stick figure for the actor and a rectangle for the system boundary.

10. DATA REQUIREMENTS

The current project has **no server-side database**. The frontend maintains logical data in JavaScript objects/arrays and, if implemented, saves the player's progress in browser localStorage as JSON.

Data group	Fields
User/Player Data	playerName, avatar, level, totalXP, currentStreak, lastActiveDate, completedChallenges (list of IDs), unlockedQuests, achievements
Challenge Data	challengeId, title, description, category, difficulty, question, type (MCQ / output / fill-in / short answer), options, expectedAnswer (accepted answers or keywords), hint, xpReward
Progress Data	completedChallengeIds, xpEarned, currentLevel, achievementStatus (earned/locked per badge), streak information (count, last date), per-quest completion counts

Storage rules: Challenge data is read-only and part of the source code. Player/progress data is the only data written, and only to localStorage under one key (for example "codequest_player"). Data is validated on load.

Future Database Scope. A backend and database could be introduced in a future version for accounts, cloud synchronization and leaderboards. **This is NOT part of the current project.**

11. SYSTEM ARCHITECTURE

User → HTML Structure → CSS Presentation Layer → JavaScript Logic Layer → Browser Local Storage (optional)

Layer	Role
User	Interacts through the browser with clicks, typing and navigation.
HTML Structure	Defines pages, sections, forms, buttons, cards and semantic landmarks.
CSS Presentation Layer	Applies theme, layout, responsiveness, states (locked/unlocked) and animations.
JavaScript Logic Layer	Handles events, challenge loading, validation, XP, levels, unlock rules, achievements, streaks, DOM updates and storage calls.

Layer	Role
Browser Local Storage (optional)	Keeps player progress between visits on the same device.

All layers execute in the browser; no server, API or database architecture exists in the current implementation.

12. TECHNOLOGY STACK

Technology	Purpose
HTML5	Structure and pages
CSS3	Styling, layout, animations, responsiveness
Vanilla JavaScript	Logic and interactivity
Figma	UI/UX design and prototype
GitHub	Version control and project hosting

No frameworks, backend technologies, databases or external APIs are used in the current scope.

13. FRONTEND MODULES

No.	Module	Responsibilities
1	Landing Module	Landing content, Start button, profile creation form, returning-player check
2	Dashboard Module	Summary cards, profile, XP/level bar, streak, continue button
3	Quest Module	Quest map rendering, category/difficulty selection, lock display
4	Challenge Module	Load challenge by ID, render question and input area, hints, submit handling
5	Validation Module	Empty/invalid checks, normalization, answer comparison, result object
6	XP Module	XP by difficulty, one-time award, XP popup data

No.	Module	Responsibilities
7	Level Module	Level formula, progress-to-next-level percentage, level-up detection
8	Progress Module	Completion counts, percentages, streak update, unlock evaluation
9	Achievement Module	Badge definitions, condition checks, earned/locked status
10	Storage Module	Save, load, validate and reset localStorage data; fallback if unavailable
11	Navigation Module	Screen switching, active links, back navigation, invalid-state redirect
12	Responsive UI Module	Media queries, mobile menu toggle, layout adjustments

14. DSA CHALLENGE STRUCTURE

Organization. Challenges are grouped Quest (category) → Difficulty → Challenge. Eight quests: **Arrays, Strings, Searching, Sorting, Linked List, Stack, Queue, Basic Algorithms**. Each quest contains Easy, Medium and Hard challenges (minimum 1 each; target 2 each = 48 challenges). Challenge types are beginner-friendly: multiple-choice, output prediction, fill-in-the-blank, and short answer/pseudocode keyword check.

Difficulty	Challenge complexity	XP reward	Unlock progression
Easy	Concept recall, simple output prediction	10 XP	Open from the start in an unlocked quest
Medium	Short logic, tracing, simple fill-in	25 XP	Unlocks after all Easy challenges of that quest are completed
Hard	Multi-step reasoning, short answer/pseudocode	50 XP	Unlocks after all Medium challenges of that quest are completed

15. XP AND LEVEL SYSTEM

XP earning. Easy 10, Medium 25, Hard 50 XP, awarded once per challenge on the first correct completion. Maximum total with 48 challenges = 8 × 2 × (10 + 25 + 50) = 1,360 XP.

Level calculation. Level = floor(totalXP / 100) + 1, maximum Level 10 (reached at 900 XP).

Level	XP required	Rank title
1	0	Novice Coder
2	100	Apprentice
3	200	Adventurer
4	300	Squire
5	400	Knight
6	500	Ranger
7	600	Mage
8	700	Champion
9	800	Hero
10	900	Code Master

Progress bar. Percentage = (totalXP mod 100) / 100 × 100 (full at Level 10).

Level unlock. Reaching a level shows a level-up message and updates the rank title.

Quest unlock. Quest 1 (Arrays) is unlocked at start. Quest N+1 unlocks when the player has completed all Easy and Medium challenges of Quest N. Difficulty tiers inside a quest unlock as defined in Section 14.

16. ACHIEVEMENT SYSTEM

Badge	Unlock condition
First Quest	Complete the first challenge
5 Challenges Completed	Total completed challenges ⩾ 5
10 Challenges Completed	Total completed challenges ⩾ 10
First Hard Challenge	Complete any Hard challenge

Badge	Unlock condition
3-Day Streak	Streak count reaches 3
Level Up	Reach Level 2 or higher
Realm Conqueror	Complete every challenge in any one quest

Each badge is awarded once, stored with progress data and shown on the Achievements page and as a notification when earned.

17. ERROR HANDLING

Situation	Handling and user-friendly message
Empty answer	Block submission: "Please enter your answer before submitting."
Invalid input	Reject disallowed/over-length input: "Please use up to 500 characters of plain text." Invalid player name: "Name must be 2-20 letters, numbers or spaces."
Incorrect answer	Show "Not quite right. Check the hint and try again!"; no XP; retry allowed
Missing challenge	"This challenge could not be found." with button back to the quest
Invalid navigation state	Redirect to dashboard (or landing if no profile) with a short notice, for example when opening a locked quest
localStorage unavailable	Run in session-only mode: "Progress cannot be saved on this browser."
Corrupt stored data	Ignore data and start with defaults after notifying the user
Browser compatibility issues	Show a notice: "Please use an updated browser for the best experience."; core pages degrade gracefully

18. SECURITY REQUIREMENTS

- Do not store passwords; do not collect sensitive personal information (only a nickname).
- Validate user input and display it as plain text (textContent) rather than inserting raw HTML (avoid unsafe innerHTML).
- Do not place secrets or sensitive information in JavaScript or comments.
- No authentication in the current scope; no payment or sensitive data processing.
- Stored progress is not encrypted and can be edited by the user.
- **A frontend-only application cannot provide strong server-side security;** challenge answers in client code can be inspected, so the app is for learning, not for graded or competitive exams.

19. TESTING REQUIREMENTS

Unit Testing. Verify JavaScript functions individually: XP calculation, level calculation, answer validation, progress calculation, streak update and unlock rules, using manual test tables or simple console checks.

Integration Testing. Verify Challenge → Validation → XP → Progress → Level → Badge flow, and Quest completion → Unlock → Next quest availability, including saving and reloading data.

UI Testing. Check buttons, navigation links, layout, cards, forms, input messages, locked/unlocked states and animations.

Responsive Testing. Test desktop (about 1366 px and above), tablet (about 768 px) and mobile (about 375 px) using browser developer tools and a real phone if possible.

Browser Testing. Test current versions of Chrome, Edge and Firefox.

Sample Test Cases

Test ID	Feature	Input	Expected Result	Status
TC-01	Landing page	Open application	Title, tagline and Start button displayed	Pending
TC-02	Start adventure	Click Start Adventure	Profile form (new) or dashboard (returning) opens	Pending

Test ID	Feature	Input	Expected Result	Status
TC-03	Profile validation	Empty name	Error message; no profile created	Pending
TC-04	Profile creation	Name "Arjun", avatar selected	Profile saved; dashboard shows Level 1, 0 XP	Pending
TC-05	Locked quest	Click Quest 2 at start	Locked message with unlock condition	Pending
TC-06	Empty answer	Submit blank answer	Prompt to enter answer; no XP	Pending
TC-07	Correct answer	Correct Easy answer	"Correct" result; +10 XP	Pending
TC-08	Incorrect answer	Wrong answer	Retry feedback; XP unchanged	Pending
TC-09	Repeat completion	Re-solve a completed challenge	No additional XP	Pending
TC-10	XP by difficulty	Solve Medium, then Hard	+25 XP, then +50 XP	Pending
TC-11	Level calculation	Total XP 99 → 100	Level changes 1 → 2 with level-up message	Pending
TC-12	Progress bar	40 XP at Level 1	Bar shows 40%	Pending
TC-13	Tier unlock	Finish all Easy in Arrays	Medium tier becomes available	Pending
TC-14	Quest unlock	Finish all Easy and Medium in Arrays	Strings quest unlocks	Pending
TC-15	Badge	Complete first challenge	"First Quest" badge earned	Pending
TC-16	Streak	Complete challenges on 3 consecutive dates	Streak = 3; "3-Day Streak" badge	Pending
TC-17	Storage reload	Earn XP, refresh page	Progress restored	Pending
TC-18	Reset progress	Click Reset, confirm	All data cleared; Level 1, 0 XP	Pending

Test ID	Feature	Input	Expected Result	Status
TC-19	Responsive layout	Open at 375 px width	Single column; hamburger menu; no horizontal scroll	Pending
TC-20	Browser compatibility	Run in Chrome, Edge, Firefox	Same behaviour in all	Pending

20. TRACEABILITY MATRIX

Requirement ID	Feature	Use Case	Test Case
FR-01	Landing Page	UC-01	TC-01, TC-02
FR-02	Player Profile	UC-01	TC-03, TC-04
FR-03	Dashboard	UC-02	TC-04
FR-04, FR-05	Quest and Category Selection	UC-03	TC-05
FR-06	Difficulty Selection	UC-03	TC-13
FR-07	Coding Challenge	UC-04	TC-06
FR-08	Challenge Validation	UC-05	TC-06, TC-07, TC-08
FR-09	XP Reward	UC-06	TC-07, TC-09, TC-10
FR-10	Level Progression	UC-07	TC-11, TC-12
FR-11	Progress Tracking	UC-08	TC-12
FR-12	Achievements	UC-09	TC-15
FR-13	Streak	UC-08	TC-16
FR-14	Locked/Unlocked Quests	UC-03, UC-07	TC-05, TC-13, TC-14
FR-15, FR-16	Completion and Result	UC-05, UC-10	TC-07, TC-08
FR-17	Navigation	UC-02, UC-10	TC-02, TC-05
FR-18	Responsive Interface	All	TC-19, TC-20
FR-19	Local Storage	UC-02	TC-17
FR-20	Reset Progress	UC-02	TC-18

21. PROJECT SCOPE

In Scope (will be developed): landing page, player profile, dashboard, quest map with 8 DSA quests, difficulty tiers, predefined challenges with answer validation, XP, levels, progress tracking, achievements, streaks, locked/unlocked content, result screen, responsive UI, optional local progress storage, reset, Figma design, GitHub repository and hosting.

Out of Scope: backend server; database server; user authentication; real-time multiplayer; cloud-based synchronization; AI coding assistant; payment system; social networking; native mobile application.

Future Enhancements (not current features): backend; database; login/signup; cloud progress synchronization; global leaderboard; multiplayer challenges; AI coding assistant; more DSA topics; real coding compiler; user-generated challenges.

22. GITHUB AND VERSION CONTROL

Area	Usage
Source code repository	One public/private repository holding HTML, CSS, JS, assets and documentation
Version control	Regular commits with clear messages (for example "Add XP calculation function")
Branch management	main (stable), develop (integration), feature branches per module (for example feature/xp-module) merged by pull request
Commit history	Evidence of individual contribution and project progress for evaluation
Collaboration	Pull requests, code review comments, shared repository access for team members
Issue tracking	GitHub Issues for tasks and bugs, labelled by module or priority
Project documentation	README with overview, setup, screenshots, Figma link; SRS stored in a docs folder
Deployment	GitHub Pages hosts the static site from the main branch (if used)

23. FIGMA REQUIREMENTS

Deliverable	Description
Low-fidelity wireframes	Black-and-white layouts of all screens
High-fidelity UI	Final coloured screens with typography, icons and states
User flow	Diagram of navigation from landing to result and back
Prototype	Clickable flow connecting main screens for demonstration and feedback
Design system	Colours, text styles, spacing, shadows, icons
Responsive layouts	Desktop, tablet and mobile frames
Components	Buttons, cards, progress bars, badges, navigation bar, input fields (with variants such as locked/unlocked)

24. DEVELOPMENT PLAN

Phase	Activity	Main output	Suggested duration
1	Requirement Analysis	This SRS	1 week
2	Figma UI/UX Design	Wireframes, UI, prototype	1-2 weeks
3	HTML Structure	All page structures	1 week
4	CSS Styling and Responsive Design	Themed, responsive UI	1-2 weeks
5	JavaScript Functionality	Navigation, events, validation	2 weeks
6	DSA Challenge Integration	Challenge data set (target 48)	1 week
7	XP/Level/Progress System	XP, levels, unlocks, badges, streaks, storage	1-2 weeks
8	Testing	Test results and bug fixes	1 week
9	GitHub Deployment	Hosted version	2-3 days

Phase	Activity	Main output	Suggested duration
10	Documentation and Presentation	Final report, demo, slides	1 week

25. ACCEPTANCE CRITERIA

The project is successful if: the user can start the adventure; navigate the dashboard; select quests; attempt challenges; have answers validated; receive XP; progress through levels; unlock challenges and quests; view progress and achievements; use the UI responsively; run the project in modern browsers; the source code is maintained on GitHub; and a UI/UX prototype exists in Figma. All 20 functional requirements must pass their acceptance criteria, and the sample test cases must pass or have documented fixes.

26. LIMITATIONS

- No server-side authentication.
- No cloud synchronization; progress exists only in the same browser/device (if storage is implemented) and is lost if browser data is cleared.
- No real-time multiplayer and no centralized database or global leaderboard (any leaderboard is mock data).
- Answers are checked by predefined answers/keywords; real code compilation and execution are not supported.
- Limited protection of client-side challenge data and progress, which users can inspect or edit.
- Limited number of predefined challenges.

27. FUTURE ENHANCEMENTS

The following are **future roadmap items only, not part of the current implementation**: (1) login/signup with a backend and database for permanent accounts; (2) cloud progress synchronization across devices; (3) global leaderboard; (4) multiplayer and timed challenge battles; (5) AI coding assistant for hints; (6) additional topics such as Trees, Graphs, Recursion and Dynamic Programming; (7) real in-browser code compiler with test cases; (8) user-generated challenges and an admin/content panel; (9) native mobile application.

28. CONCLUSION

CodeQuest: RPG Coding Adventure combines coding education with RPG-style gamification in a realistic, frontend-only web application built with HTML5, CSS3, Vanilla JavaScript, Figma and GitHub. By rewarding every solved DSA challenge with XP, levels, badges, streaks and unlocked quests, it motivates learners to practise regularly and makes progress visible. This SRS defines the scope, requirements, design, testing and limitations needed to implement, evaluate and later extend the project, while keeping the current version simple, achievable and professionally documented.